第7章 第7节 Resources + Tools + Prompts 组合

- 来源: https://class.imooc.com/lesson/5280#mid=104936
- 阶段: 阶段三: 【工具与协议】让AI连接世界
- 周: 第7周 MCP：模型上下文协议
- 模块: Tools、Prompts详解
- 课节: 图文 1-3 Resources + Tools + Prompts 组合

1-3 Resources + Tools + Prompts 组合

第7章 第7节 Resources + Tools + Prompts 组合

> [!TIP] 阅读指南 前面三节分别学习了 Resources、Tools 和 Prompts 各自的用法。本节通过一个完整的智能客服系统场景，展示三者如何在实际项目中协同工作。

7.1 Resources + Tools + Prompts 组合使用

实际上，这三者可以混合使用：

```
【MCP Server】
       +-------------------------+
       |
       |  1. Resources (读)
       |     +- 数据源列表
       |
       |  2. Tools (写)
       |     +- 可执行操作
       |
       |  3. Prompts (模板)
       |     +- 使用指南
       |
       +-------------------------+
                   ↓
           统一的 MCP 协议
                   ↓
             AI 应用 (Client)
```

预览

场景1：智能客服系统

一个客服 MCP Server，能查询订单、处理退款、发送通知。

下面是它的完整设计——注重函数名和每层职责，不展开实现细节：

```
class CustomerServiceMCPServer:
    """
    智能客服 MCP Server：
    通过三层接口（Resources / Tools / Prompts）把客服能力暴露给 AI 应用
    """

    # ═══════════════════════════════════════
    #  Resources：只读知识库
    #  AI 调用 list_resources() 拿到列表，
    #  再按需 read_resource(uri) 获取具体内容
    # ═══════════════════════════════════════
    def list_resources(self):
        return [
            {"uri": "kb://faq",     "description": "常见问题标准答案"},
            {"uri": "kb://policy",  "description": "退换货政策（时效/条件/流程）"},
            {"uri": "kb://contact", "description": "客服联系方式"}
        ]

    # ═══════════════════════════════════════
    #  Tools：可执行操作（有副作用）
    #  AI 调用 list_tools() 拿到列表，
    #  再按需 call_tool(name, args) 触发实际操作
    # ═══════════════════════════════════════
    def list_tools(self):
        return [
            {"name": "query_order",       "description": "按订单号查询状态、金额、物流"},
            {"name": "process_refund",    "description": "发起退款，修改订单状态为已退款"},
            {"name": "send_notification", "description": "向用户发送短信/邮件通知"}
        ]

    # ═══════════════════════════════════════
    #  Prompts：预制工作流模板
    #  把 Resource + Tool 组合成固定流程，
    #  每个 Prompt 内含操作指引——告诉 AI 该先后做什么
    # ═══════════════════════════════════════
    def list_prompts(self):
        return [
            {"name": "handle_refund_request", "description": "退款处理全流程"},
            {"name": "order_inquiry",         "description": "订单查询与物流追踪"},
            {"name": "complaint_handling",    "description": "投诉受理与升级"}
        ]
```

光看接口定义有点抽象，跟着一个真实请求走一遍，立刻就能明白三层是怎么协作的。

一个退款请求的完整旅程

光看接口定义有点抽象。下面跟着一个真实退款请求，看它是怎么在三方之间流转的。

这三个角色的职责先明确一下：LLM 是大脑——负责理解用户意图、做判断、决定下一步该调什么；AI 应用 是调度者——LLM 决定之后，AI 应用通过协议向 MCP Server 发请求、收回结果、喂给 LLM，它自己不碰磁盘也不执行具体的业务操作；MCP Server 是执行者——负责提供 Resources / Tools / Prompts 的能力列表，并且在被 AI 应用调用时，真正执行实际操作（读文件、查数据库、发短信、退款处理等）。LLM 从不直接调 MCP Server，每一步都是 LLM 决策、AI 应用调度、MCP Server 执行。

```
用户：「我要退货，订单号 12345」
  │
  ▼

── 第一步：意图识别 → 匹配 Prompt ──────────────────────

  │
  │  【AI 应用】 收到用户文字
  │            · 自己没有理解能力——AI 应用只是一段程序代码，看不懂自然语言
  │            · 只能把「我要退货，订单号 12345」原封不动转发给 LLM
  │
  ▼
  【LLM】 收到文字，开始理解：
        · 关键词「退货」→ 这是一个退款请求
        · 关键词「订单号 12345」→ 需要处理具体订单
        · 结论：需要走退款处理流程

        但 LLM 不知道这个 MCP Server 注册了哪些能力——
        有没有退款相关的 Prompt？可以用哪些 Tool？能查阅哪些 Resource？
        LLM 必须通过 MCP Server 暴露的接口列表才能知道答案。
        → 先查 Prompt 模板：指示 AI 应用调 list_prompts()
  │
  │  list_prompts()
  ▼
  【MCP Server】 返回注册的全部 Prompt：
        · **handle_refund_request** — 退款处理全流程
        · **order_inquiry**         — 订单查询与物流追踪
        · **complaint_handling**    — 投诉受理与升级
  │
  │  返回 Prompt 列表
  ▼
  【LLM】 逐一比对每个 Prompt 的描述文字——做的是语义匹配，不是精确关键词匹配：
        ·「退款处理全流程」→ 和用户说的「退货」语义最接近
        ·「订单查询」       → 用户没说要查订单，排除
        ·「投诉处理」       → 用户没说要投诉，排除
        → 选定 **handle_refund_request**
        → 指示 AI 应用：拿这个 Prompt 内嵌的操作指引
  │
  ▼

── 第二步：获取操作指引 ──────────────────────────────

  │
  │  get_prompt("handle_refund_request")
  ▼
```

```
【MCP Server】 Prompt 不仅是个名字，内部预置了完整的操作流程——
      先做什么、后做什么、每一步调哪个接口，全都写好了：
      ·
      ·  **1. query_order(12345)**
      ·     → 确认订单存在，拿到金额和状态
      ·  **2. 读 kb://policy Resource**
      ·     → 获取退换货政策，判断是否可退
      ·  **3. process_refund(order_id, amount)**
      ·     → 执行退款
      ·  **4. send_notification(user, msg)**
      ·     → 发短信告知用户退款结果
      ·
  │
  │  返回操作指引
  ▼
【LLM】 拿到指引，LLM 才知道具体该怎么走
      → 指示 AI 应用：「按指引，从第一步开始执行」
  │
  ▼

━━ 第三步：按指引逐步执行 ━━━━━━━━━━━━━━━━━━━━━━━━

  ── Step 1：查订单 ──

  【LLM】 决定：需要先看这个 MCP Server 提供了哪些 Tool
      → 指示 AI 应用：调 list_tools()
  │
  │  list_tools()
  ▼
【MCP Server】 返回：query_order、process_refund、send_notification
  │
  │  返回 Tool 列表
  ▼
【LLM】 在列表中找到 **query_order**，决定用订单号 12345 查询
      · 需要确认：订单是否存在？金额多少？什么时候签收的？
      · 这些信息决定了后续能不能退款
      → 指示 AI 应用：call_tool("query_order", 12345)
  │
  │  call_tool("query_order", 12345)
  ▼
【MCP Server】 查询数据库，返回：
      订单 ¥299，状态「已签收」，签收 3 天前
  │
  │  返回查询结果
```

```
▼
【LLM】 拿到订单数据

── Step 2：读政策 ──

【LLM】 决定：光有订单数据不够，还需要知道退货规则
      · 什么条件下能退？几天之内有效？
      · 不知道规则就没法判断能不能退
      → 指示 AI 应用：read_resource("kb://policy")
│
│  read_resource("kb://policy")
▼
【MCP Server】 返回政策内容：「签收 7 天内可无理由退货」
│
│  返回政策内容
▼
【LLM】 拿到退货政策

── Step 3：判断 + 执行退款 ──

【LLM】 现在手里有两组数据，可以做出判断了：
      · 订单签收日期 = 3 天前
      · 退货政策     = 签收 7 天内可退
      ·
      这一步只有 LLM 能做——MCP Server 只负责提供数据和执行指令，
      不参与任何决策，推理和判断完全是 LLM 的事。
      → 推理：3 天 < 7 天 → **符合退货条件，可以退款**
      → 指示 AI 应用：call_tool("process_refund",
                                order_id=12345,
                                amount=299)
│
│  call_tool("process_refund", order_id=12345, amount=299)
▼
【MCP Server】 执行退款操作，修改订单状态
      → 退款成功，款项 3-5 个工作日到账
│
│  返回退款结果
▼
【LLM】 退款成功

── Step 4：通知用户 ──

【LLM】 决定：退款已经成功，但用户还不知道
      · 不能悄悄退款不吭声，用户需要知道处理结果
```

```
        → 指示 AI 应用：call_tool("send_notification",
                        user="138****",
                        msg="退款 ¥299 已处理")
  │
  │  call_tool("send_notification", ...)
  ▼
  【MCP Server】 发送短信 → 短信已送达
  │
  │  返回发送结果
  ▼
  【LLM】 通知已送达
  │
  ▼

── 最终回复 ──────────────────────────────────────
  │
  ▼
  【LLM】 不是简单地把前面的结果拼接起来——
        LLM 要把每一步的信息组织成用户能看懂的自然语言：
        · 订单 12345 查到了
        · 退货政策判断通过了
        · 退款已执行成功
        · 短信已发送
        →
        「您的订单 12345 已成功退款 ¥299，
          款项预计 3-5 个工作日到账，请留意短信通知。」
  │
  ▼
回复用户
```

回顾整个流程，每走一步的模式都一样：LLM 想清楚下一步该做什么 → 告诉 AI 应用去调哪个接口 → AI 应用调 MCP Server 拿到数据或触发操作 → 把结果拿回给 LLM → LLM 看了结果再想下一步。LLM 全程不动手但每一步都是它决策的，AI 应用负责编排调度——在 LLM 和 MCP Server 之间传递请求和结果，MCP Server 是真正动手的那个——被调用时执行实际操作（读文件、查数据库、退款处理、发短信等），然后把结果返回。

7.2 下一节预告

到这里，已经集齐了 MCP 的三张核心「王牌」：让 AI 能读数据的 Resources，让 AI 动起来的 Tools，以及让流程标准化的 Prompts。这些能力是如何在 AI 应用与后端服务之间精准传递，并保持跨

平台、跨语言的丝滑协作的呢？下一节将揭开 MCP 通信协议与架构的神秘面纱，深入 JSON-RPC 的底层逻辑，看透这场高效协作背后的技术真相。

7.3 ■ 学点英语

中文	English	音标	说明
智能编排	Intelligent Orchestration	/ɪnˈtelɪdʒənt ˌɔːrkɪˈstreɪʃn/	LLM智能决策+AI应用调度编排的协作模式
逐步骤决策	Step-by-Step Decision	/step baɪ step dɪˈsɪʒn/	LLM每步基于上一步结果动态决策下一步的机制
信息整合	Information Synthesis	/ˌɪnfərˈmeɪʃn ˈsɪnθəsɪs/	LLM将Server返回的多步数据整合为连贯自然语言的能力
安全闸门	Security Gate	/sɪˈkjʊrəti geɪt/	AI应用在编排过程中插入的权限校验和安全拦截机制